

Instituto Politécnico Nacional
Escuela Superior de Cómputo



Ingeniería en Sistemas Computacionales · Plan 2020

Unidad de aprendizaje: **Teoría de la Computación**

Práctica 3

Uso de JFLAP para Convertir AFD a Expresiones Regulares y Extensión del Software con AFN, AFN-λ y Minimización

Alumno: Rico Espluga Juan Diego

Boleta: 2025630474

Grupo: 4CV4

Docente: Gabriel Hurtado Avilés

Fecha de entrega: 4 de noviembre de 2026

Índice

1. Objetivo **3**

2. Ejercicio 1. Conversión de AFD a expresiones regulares **3**

2.1. Conversión con JFLAP . . . 3

2.1.1. Los cinco autómatas . . . 3

2.1.2. Estados eliminados . . . 4

2.1.3. Verificación . . . 6

2.1.4. Simplificación . . . 7

2.2. Comparación de métodos . . . 7

2.2.1. AFD 142a: (ab)* . . . 7

2.2.2. AFD 138: número de b múltiplo de 3 . . . 8

2.2.3. Comparación . . . 9

3. Ejercicio 2. Extensión del software interactivo **9**

3.1. Cerradura λ y simulación directa . . . 10

3.2. Eliminación de transiciones λ . . . 10

3.3. Construcción de subconjuntos . . . 12

3.4. Minimización y equivalencia . . . 12

3.5. Validación de varias cadenas . . . 13

3.6. Pruebas automatizadas . . . 14

4. Ejercicio 3. Documentación de los algoritmos **15**

4.1. Cerradura λ y simulación directa . . . 16

4.2. Eliminación de transiciones λ . . . 17

4.3. Construcción de subconjuntos . . . 17

4.4. Minimización . . . 18

4.5. Validación por lotes . . . 19

5. Conclusiones **19**

1. Objetivo

Relacionar los autómatas finitos con las expresiones regulares mediante JFLAP e incorporar al software propio los algoritmos que transforman un AFN con transiciones λ en un AFD mínimo.

La práctica tiene tres ejercicios. El primero se resuelve en JFLAP y a mano; el segundo extiende el software de la Práctica 2 y el tercero documenta sus algoritmos. Los autómatas están en `automatas/practica3/` y el reporte en Markdown en `docs/practica3/`.

2. Ejercicio 1. Conversión de AFD a expresiones regulares

2.1. Conversión con JFLAP

2.1.1. Los cinco autómatas

Se eligieron cinco AFD de la Lista 2 con estructuras distintas: dos con estado sumidero (119 y 142a), uno con alfabeto de tres símbolos (122), un contador módulo 3 (138) y un producto de dos contadores módulo 2 (140b). Sus estados se renombraron q0, q1, … en orden de recorrido para que coincidan con los nombres que JFLAP asigna; los archivos están en `automatas/practica3/afd-a-er/`.

Tabla 1. Análisis previo.

AFD	Lenguaje	Ciclos y ramas que anticipan la expresión
119	Sobre {a, b}: después de cada `aa` aparece una `b`	Todo regresa a q0 con `b`; desde q0 se sale con `a` (q1) o `aa` (q2) y se vuelve con `b`. El sumidero q3 no aporta términos. Se anticipa (b + ab + aab)* seguida de un final opcional `a`.
122	Sobre {0, 1, 2}: todo bloque de tres símbolos tiene un 0	Igual estructura con 0 como «regreso» y 1, 2 como avance; dos finales extra (q1, q2) dan la cola opcional.
138	Sobre {a, b}: el número de `b` es múltiplo de 3	Un ciclo de tres `b` con lazos `a` en cada estado: (a + ba*ba*b)*.
140b	Sobre {0, 1}: número par de 0 y par de 1	Cuatro estados en un ciclo de paridades; los pares `00`, `11` regresan y `01`, `10` cruzan.
142a	(ab)*	Un solo ciclo q0 → q1 → q0 con `ab`; q2 es sumidero.

2.1.2. Estados eliminados

JFLAP (*Convert* → *Convert FA to RE*) agrega un estado final único con transiciones λ, une las transiciones paralelas y pide eliminar los estados uno por uno; al quitar k, cada par de aristas p → k → q se reemplaza por $R_{pq} + R_{pk}R_{kk}^{*}R_{kq}$ (Rodger & Finley, 2006, cap. 3). Conviene eliminar primero el sumidero (no tiene camino al final, así que desaparece sin dejar términos) y después los estados con menos aristas entrantes y salientes. JFLAP no deja eliminar el estado inicial ni el final que agrega: al terminar quedan q0 y el final nuevo T (JFLAP lo nombra q4), y la expresión se lee como $(R_{00})^{*}R_{0T}$. La tabla documenta cada eliminación tal como la hizo JFLAP; la última columna es la expresión que JFLAP muestra al terminar, con espacios alrededor de + para que se pueda leer.

Tabla 2. Eliminación de estados en JFLAP.

AFD	Orden	Lo que queda después de cada eliminación	Expresión de JFLAP
119	q3, q2, q1	q3: nada (es sumidero). q2: R(q1,q0) = b + ab. q1: R(q0,q0) = b + a(b + ab), R(q0,T) = λ + a.	(b + a(b + ab))*(λ + a)
122	q3, q2, q1	q3: nada. q2: R(q1,q0) = 0 + (1 + 2)0, R(q1,T) = λ + 1 + 2. q1: R(q0,q0) = 0 + (1 + 2)(0 + (1 + 2)0), R(q0,T) = λ + (1 + 2)(λ + 1 + 2).	(0 + (1 + 2)(0 + (1 + 2)0))*(λ + (1 + 2)(λ + 1 + 2))
138	q1, q2	q1: R(q0,q2) = ba*b. q2: R(q0,q0) = a + ba*ba*b, R(q0,T) = λ.	(a + ba*ba*b)*
140b	q1, q2, q3	q1, q2: R(q0,q0) = 00 + 11, R(q0,q3) = 01 + 10, R(q3,q0) = 10 + 01, R(q3,q3) = 11 + 00. q3: R(q0,q0) = 00 + 11 + (01 + 10)(11 + 00)*(10 + 01).	(00 + 11 + (01 + 10)(11 + 00)*(10 + 01))*
142a	q2, q1	q2: nada. q1: R(q0,q0) = ab, R(q0,T) = λ.	(ab)*

El orden importa: en JFLAP, eliminar 140b en el orden q3, q2, q1 da (1(00)*1 + (0 + 1(00)*01)(11 + 10(00)*01)*(0 + 10(00)*1))*, y en el software del Ejercicio 2, que sí puede quitar q0 porque agrega también un inicial I, eliminar en el orden de Q (q0 primero) produce para 140b una expresión de 234 caracteres y para 122 una de 145, aunque todas describen los mismos lenguajes.

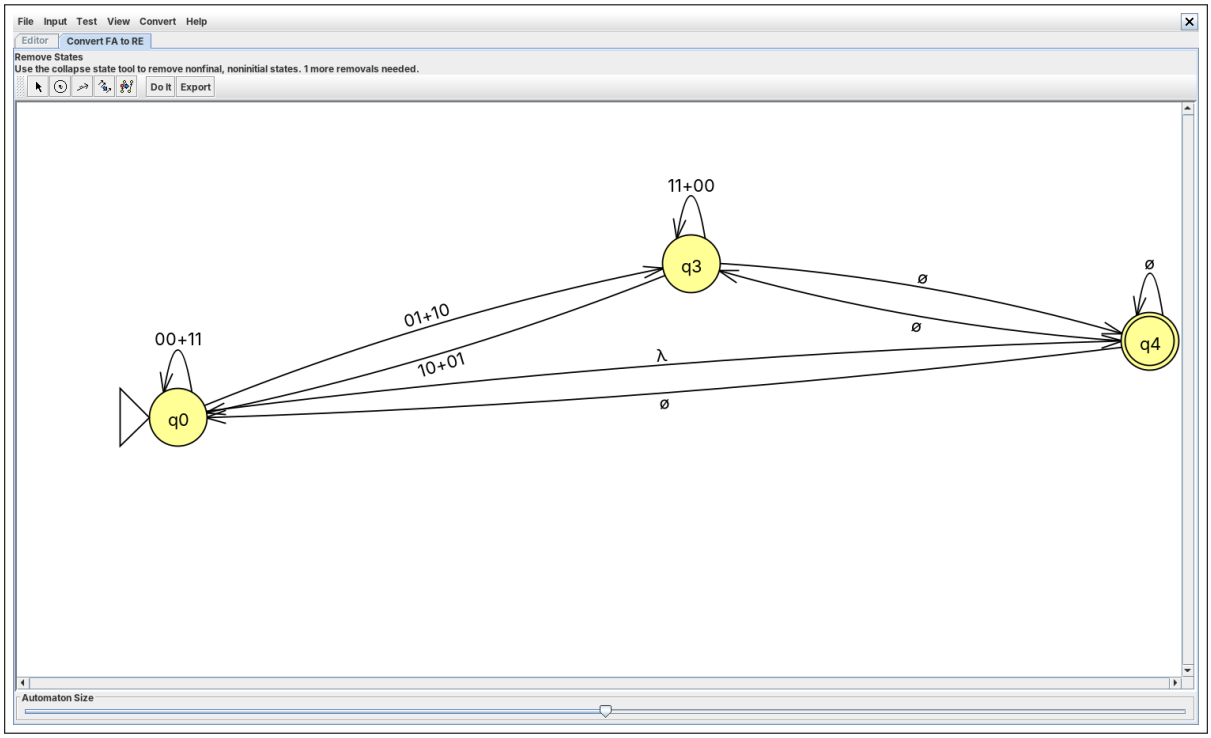


Figura 1. JFLAP: Convert FA to RE sobre el AFD 140b, después de eliminar q1 y q2.

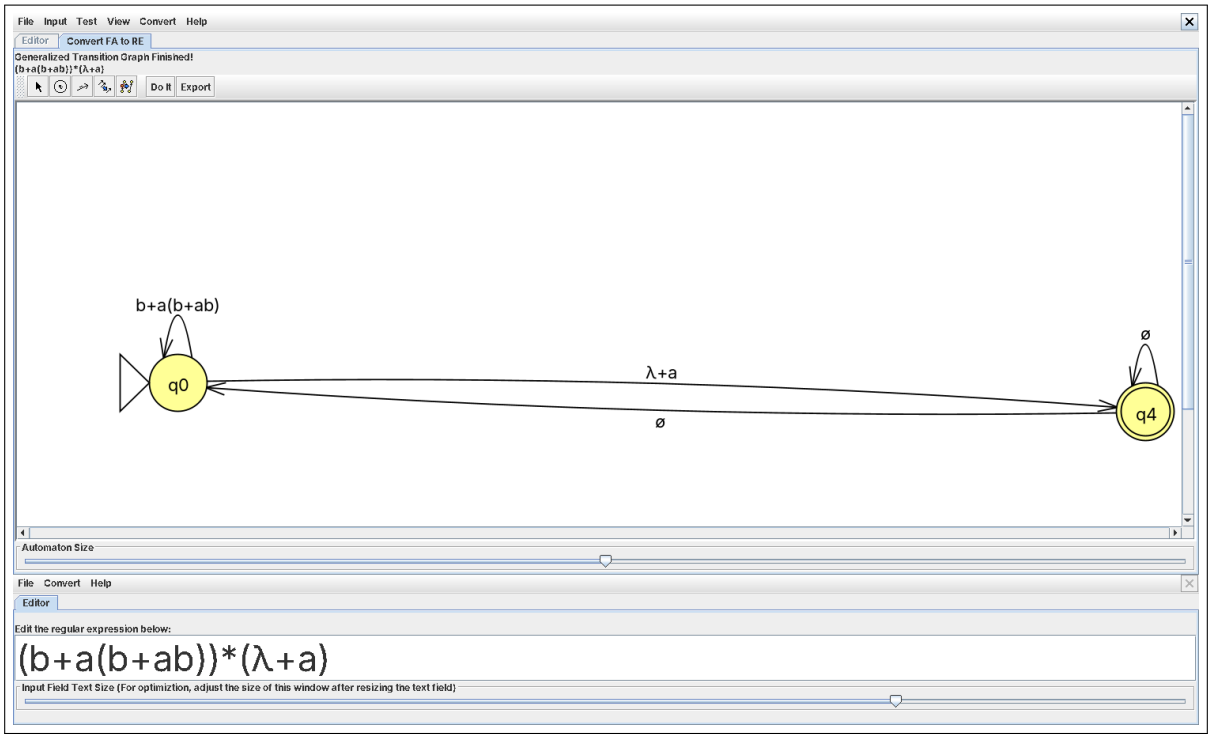


Figura 2. JFLAP: expresión final del AFD 119.

2.1.3. Verificación

Cada expresión se volvió a convertir en autómata con *Convert to NFA* y se comparó con el AFD original con *Test* → *Compare Equivalence*; en las cinco JFLAP responde que son equivalentes. El software del Ejercicio 2 confirma lo mismo con la prueba de equivalencia por AFD mínimos. Además se probaron cinco cadenas en ambos formalismos:

Tabla 3. Cadenas de prueba.

AFD	Cadenas (✓ aceptada, ✗ rechazada; mismo resultado en el AFD y en la expresión)
119	λ ✓, `a` ✓, `ababa` ✓, `aa` ✗, `abbaa` ✗
122	λ ✓, `100` ✓, `22010` ✓, `112` ✗, `0122` ✗
138	λ ✓, `aaa` ✓, `aabbb` ✓, `ba` ✗, `abba` ✗
140b	λ ✓, `0000` ✓, `01100110` ✓, `10` ✗, `0100` ✗
142a	λ ✓, `abab` ✓, `abababababab` ✓, `aa` ✗, `abba` ✗

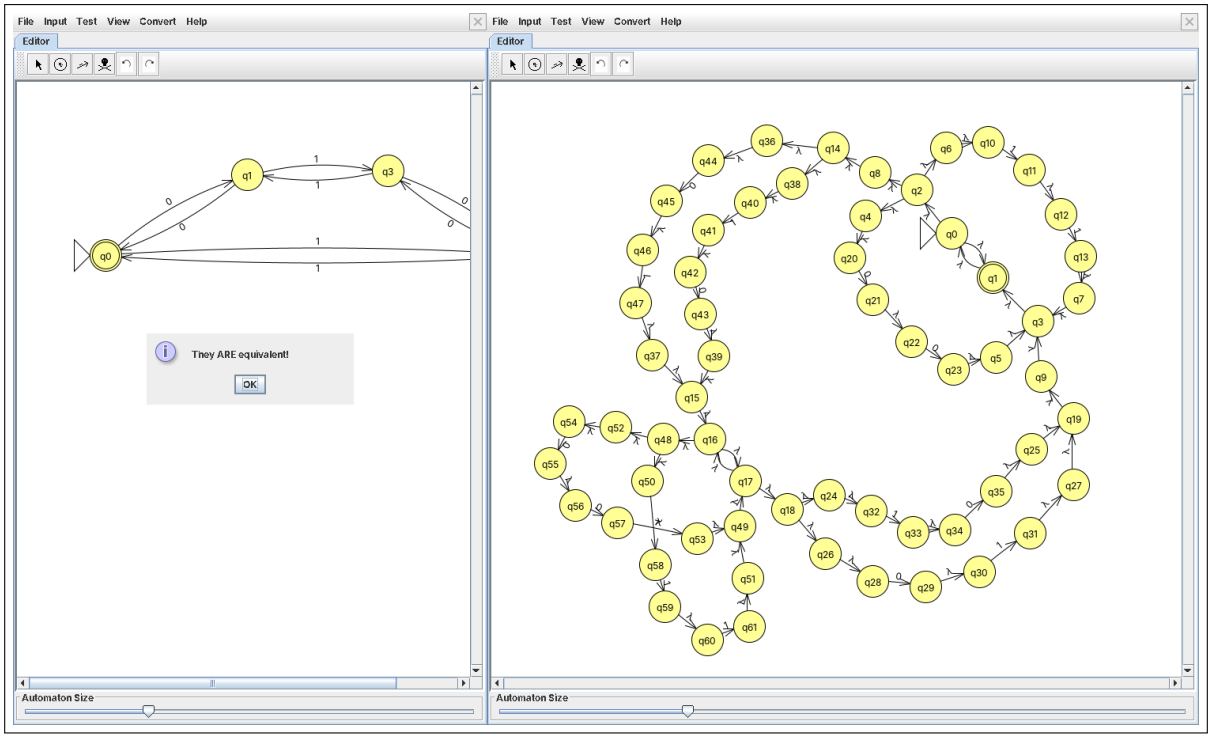


Figura 3. JFLAP: Compare Equivalence entre el AFD 140b y el autómata de su expresión.

2.1.4. Simplificación

Las leyes algebraicas que se usan son las de Hopcroft et al. (2007, sec. 3.4): conmutatividad y asociatividad de +, idempotencia (R + R = R), distributividad (R(S + T) = RS + RT), $\varnothing$ como neutro de + y absorbente de la concatenación, λ como neutro de la concatenación, (R*)* = R*, λ + RR* = R* y la ley de desplazamiento R(SR)* = (RS)*R.

- **119.** b + a(b + ab) = b + ab + aab por distributividad, así que la expresión es (b + ab + aab)*(λ + a): bloques que terminan en `b` con a lo sumo dos `a` antes, y al final, opcionalmente, una `a` suelta. Es la que se anticipó en el análisis.
- **122.** Igual que 119, con 0 en el papel de `b` y (1 + 2) en el de `a`: (0 + (1 + 2)0 + (1 + 2)(1 + 2)0)*(λ + (1 + 2) + (1 + 2)(1 + 2)).
- **138** y **142a** ya están en su forma más simple; en 142a, la expresión que resulta de eliminar en el orden de Q es λ + a(ba)*b, y por desplazamiento a(ba)*b = (ab)*ab, de modo que λ + (ab)*ab = λ + $(ab)^+$ = (ab)*.
- **140b.** Por conmutatividad, 11 + 00 = 00 + 11 y 10 + 01 = 01 + 10; queda la forma clásica (00 + 11 + (01 + 10)(00 + 11)*(01 + 10))*.

2.2. Comparación de métodos

Para 142a y 138 la expresión se obtuvo a mano por tres métodos.

*2.2.1. AFD 142a: (ab)**

Eliminación de estados. Ya documentada: tres pasos, uno por estado, con resultado (ab)*.

Expresiones $R_{ij}^{(k)}$ (Hopcroft et al., 2007, sec. 3.2.1). Con q0 = 1, q1 = 2, q2 = 3, $R_{ij}^{(k)}$ describe los caminos de i a j que sólo pasan por estados intermedios $\leq$ k, y $R_{ij}^{(k)} = R_{ij}^{(k-1)} + R_{ik}^{(k-1)} \left(R_{kk}^{(k-1)}\right)^* R_{kj}^{(k-1)}$.

Tabla 4. $R_{ij}^{(k)}$ para 142a, simplificadas en cada paso.

	R11	R12	R13	R21	R22	R23	R31	R32	R33
k = 0	λ	a	b	b	λ	a	$\varnothing$	$\varnothing$	λ + a + b

	R11	R12	R13	R21	R22	R23	R31	R32	R33
k = 1	λ	a	b	b	λ + ba	a + bb	$\varnothing$	$\varnothing$	λ + a + b
k = 2	λ + a(ba)*b	a(ba)*	b + a(ba)*(a + bb)	(ba)*b	(ba)*	(ba)*(a + bb)	$\varnothing$	$\varnothing$	λ + a + b

Como F = {q0}, la expresión es $R_{11}^{(3)} = R_{11}^{(2)} + R_{13}^{(2)}\left(R_{33}^{(2)}\right)^* R_{31}^{(2)}$ = λ + a(ba)*b, porque $R_{31}^{(2)}$ = $\varnothing$ anula el segundo término. Por desplazamiento, λ + a(ba)*b = (ab)*.

Lema de Arden (Arden, 1961; Schreiner, 2013). Con una ecuación por estado, $X_i = \sum_a a\, X_{\delta(i,a)}$ (más λ si i $\in$ F):

X0 = aX1 + bX2 + λ, X1 = bX0 + aX2, X2 = (a + b)X2.

La última tiene la forma X = AX + B con B = $\varnothing$, así que X2 = (a + b)*$\varnothing$ = $\varnothing$. Queda X1 = bX0, y al sustituir, X0 = abX0 + λ; por Arden, X0 = (ab)*λ = (ab)*.

2.2.2. AFD 138: número de b múltiplo de 3

Eliminación de estados (q1, q2, q0): (a + ba*ba*b)*.

$R_{ij}^{(k)}$**.** Con q0 = 1, q1 = 2, q2 = 3: $R_{11}^{(0)} = R_{22}^{(0)} = R_{33}^{(0)}$ = λ + a, $R_{12}^{(0)} = R_{23}^{(0)} = R_{31}^{(0)}$ = b y las demás $\varnothing$. Simplificando con (λ + a)* = a*:

- k = 1: R11 = a*, R12 = a*b, R31 = ba*, R32 = ba*b, R33 = λ + a.
- k = 2: R13 = a*b·a*·b = a*ba*b, R33 = λ + a + ba*ba*b, R11 = a*, R31 = ba*.
- k = 3: $R_{11}^{(3)}$ = a* + a*ba*b(a + ba*ba*b)*ba*.

La equivalencia con (a + ba*ba*b)* se sigue de (R + S)* = R*(SR*)* con R = a y S = ba*ba*b: ambas describen palabras formadas por bloques de `a` y grupos de exactamente tres `b`. La prueba de equivalencia del software lo confirma.

Lema de Arden. X0 = aX0 + bX1 + λ, X1 = aX1 + bX2, X2 = aX2 + bX0. Por Arden, X2 = a*bX0; sustituyendo, X1 = aX1 + ba*bX0 $\Rightarrow$ X1 = a*ba*bX0; y X0 = aX0 + ba*ba*bX0 + λ = (a + ba*ba*b)X0 + λ $\Rightarrow$ X0 = (a + ba*ba*b)*.

2.2.3. Comparación

Tabla 5. Comparación de los tres métodos.

Método	Longitud (142a / 138)	Pasos	Facilidad	Base
Eliminación de estados	5 / 14 caracteres	uno por estado	La más visual; el orden decide el tamaño.	Hopcroft et al. (2007, sec. 3.2.2); Rodger y Finley (2006)
$R_{ij}^{(k)}$	11 / 28 sin simplificar	n³ expresiones (27 en ambos)	Mecánica pero larga; obliga a simplificar en cada paso.	Hopcroft et al. (2007, sec. 3.2.1)
Lema de Arden	5 / 14	una sustitución por estado	Algebraica y corta si se despeja en buen orden.	Schreiner (2013, sec. 2.4, teorema 4); Arden (1961)

Los tres dan expresiones equivalentes, como garantiza el teorema de Kleene, pero de forma distinta. $R_{ij}^{(k)}$ es el método que se usa para *demostrar* el teorema, porque es uniforme, pero a mano es el más largo; la eliminación de estados y Arden son el mismo cálculo visto como grafo o como sistema de ecuaciones, y ambos dan la expresión más corta cuando se eliminan primero el sumidero y los estados con menos conexiones.

3. Ejercicio 2. Extensión del software interactivo

El núcleo gana el módulo `src/nucleo/algoritmos.py`, en Python puro y sin dependencias de Flet, y la interfaz tres secciones: **Transformaciones**, **Equivalencia** y **Validación por lotes**. Todos los algoritmos comparten una misma forma: una clase que recibe el autómata y un método `resolver()` que devuelve un `Resultado` con la entrada, la salida, un resumen y la lista de `Etapa` (título, explicación, tabla y, si corresponde, el autómata parcial). La interfaz no sabe nada de cada algoritmo; sólo recorre etapas con un visor común de «Anterior / Siguiente» y una barra de avance. Así cada algoritmo se ejecuta y se prueba sin abrir la interfaz.

Tabla 6. Algoritmos de la Práctica 3.

Clase	Entrada → salida	Etapas que muestra
`QuitarLambda`	AFN-λ → AFN	Cerraduras; Δ'(q, a) con los destinos añadidos; estados que pasan a F; AFN resultante.
`Subconjuntos`	AFN → AFD	Una fila por subconjunto procesado, con los nuevos que aparecen; subconjuntos de aceptación; AFD.
`Minimizacion`	AFD → AFD mínimo	Inaccesibles; complemento con sumidero si δ es parcial; tabla de pares en cada ronda; clases; AFD mínimo.
`HaciaMinimo`	cualquiera → AFD mínimo	Encadena sólo los pasos necesarios.
`Equivalencia`	dos autómatas → veredicto	Los dos mínimos, la correspondencia de estados o el contraejemplo más corto.
`Lote`	autómata + texto → filas	Una fila por palabra; informe exportable.

Todos trabajan con autómatas importados de `.jff`: la Biblioteca los abre y los deja como autómata activo, y las demás secciones parten de él.

3.1. Cerradura λ y simulación directa

`Automata.cierre(S)` calcula λ-cl(S) y `cierres()` la de cada estado. El **Simulador** de la Práctica 2 ya simulaba los AFN-λ sin convertirlos; en esta práctica se agregó a su lado una tarjeta con la cerradura de cada estado y la traza separa, en cada paso, el conjunto que se alcanza al leer el símbolo y el que queda después de cerrarlo. Los ejercicios 196 a 200 de la Lista 3 se usan como casos de prueba.

3.2. Eliminación de transiciones λ

`QuitarLambda` aplica Δ'(q, a) = λ-cl(δ(λ-cl(q), a)) y F' = $\{q : \lambda\text{-cl}(q) \cap F \neq \varnothing\}$. La segunda etapa muestra la tabla de Δ' y, entre paréntesis, los destinos que no estaban en δ(q, a); la tercera, los estados que se vuelven de aceptación y por qué (qué estado final contiene su cerradura).

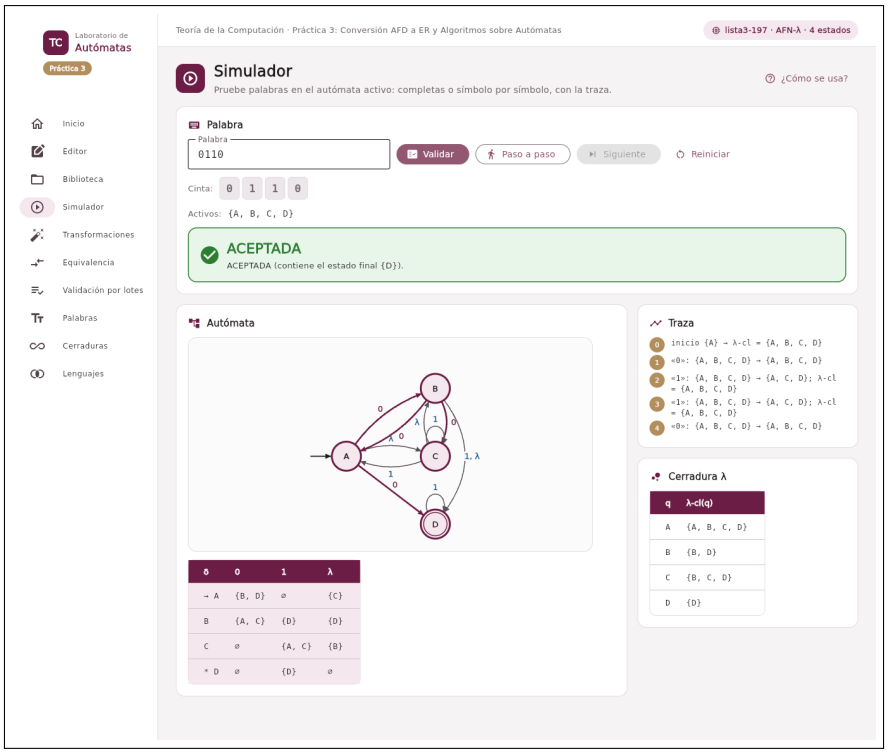


Figura 4. Simulación directa del AFN-λ 197 con la tabla de cerraduras: cada paso muestra δ(S, a) y su cierre λ.

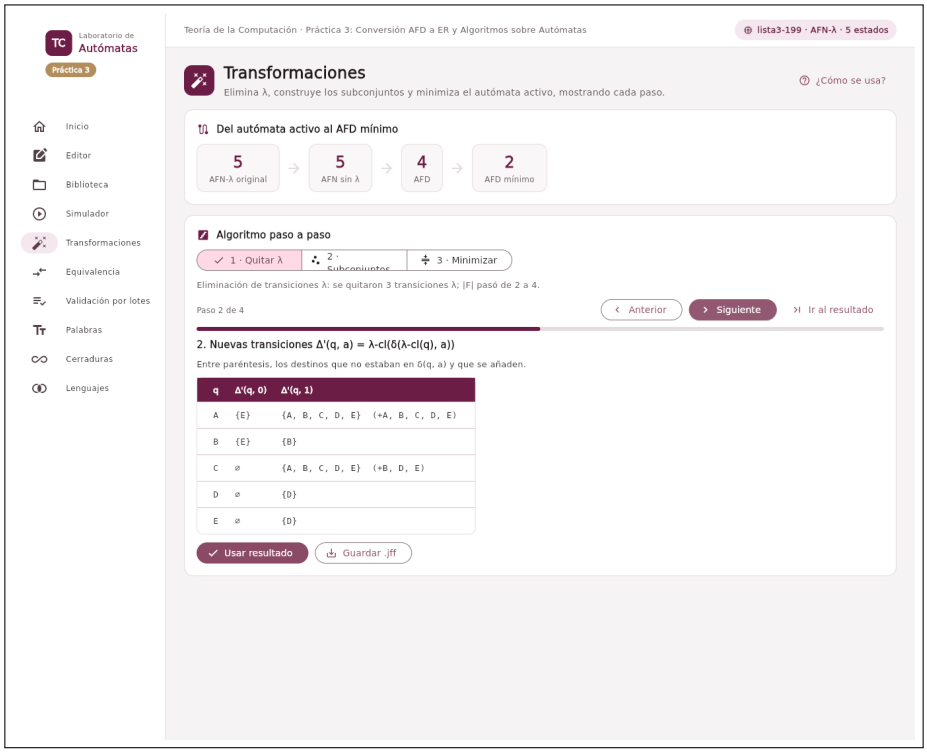


Figura 5. Transformaciones, etapa «Quitar λ» del AFN-λ 199: transiciones añadidas.

3.3. Construcción de subconjuntos

`Subconjuntos` parte de λ-cl($\{q_0\}$) y procesa una cola: por cada subconjunto y cada símbolo calcula λ-cl(δ(S, a)) y lo agrega si es nuevo. Sólo existen, por tanto, los subconjuntos **alcanzables**. Cada subconjunto recibe una letra (A, B, …) y la tabla muestra ambos («B = {q0, q2}»), de modo que el AFD resultante tiene nombres cortos sin perder la información. Una etapa por fila deja ver la tabla **conforme se genera**, y la penúltima explica qué subconjuntos son de aceptación: los que contienen algún estado de F. Por omisión no se agrega el subconjunto $\varnothing$ (como JFLAP); el parámetro `incluir_vacio` lo agrega. Casos de prueba: Lista 3, ejercicios 192 a 194 y 260 a 263 (el 191 no tiene autómata propio en la lista).

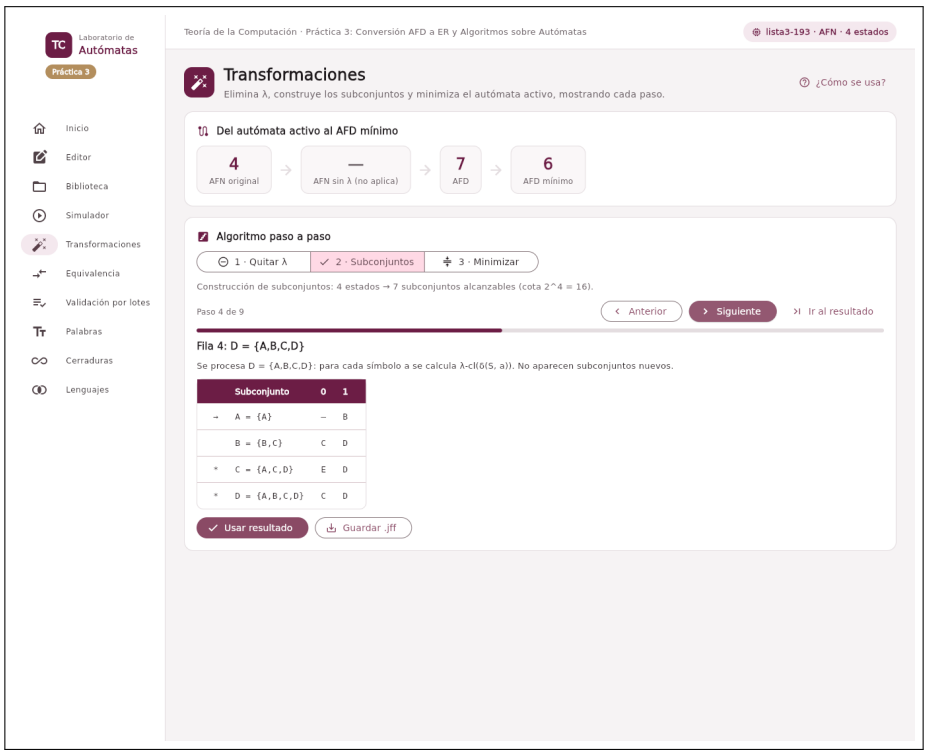


Figura 6. Construcción de subconjuntos del AFN 193, a la mitad de la tabla.

3.4. Minimización y equivalencia

`Minimizacion` usa la **tabla de pares distinguibles**. Primero elimina los inaccesibles con un recorrido en anchura; si δ es parcial agrega un sumidero para que todo par tenga sucesor. En la ronda 0 marca los pares {p, q} con exactamente un estado final; en la ronda k marca {p, q} si

para algún símbolo los sucesores forman un par marcado antes. Cada ronda se muestra como la tabla triangular, con $\times$k en los pares marcados en la ronda k y $\cdot$ en los que siguen sin marcar, y la explicación dice qué par se marcó y con qué símbolo. Al estabilizarse, los pares sin marcar forman las clases de equivalencia. Si se agregó el sumidero, su clase se retira al final para que el resultado nunca tenga más estados que la entrada. Casos de prueba: Lista 3, ejercicios 151 a 157.

La sección **Equivalencia** lleva los dos autómatas a AFD mínimos completos sobre la unión de sus alfabetos y busca un isomorfismo recorriéndolos en paralelo desde los iniciales; son equivalentes si los mínimos coinciden salvo el nombre de los estados, y en ese caso se muestra la correspondencia. Si no, una búsqueda en anchura sobre el producto da la **palabra más corta** que acepta uno y rechaza el otro.

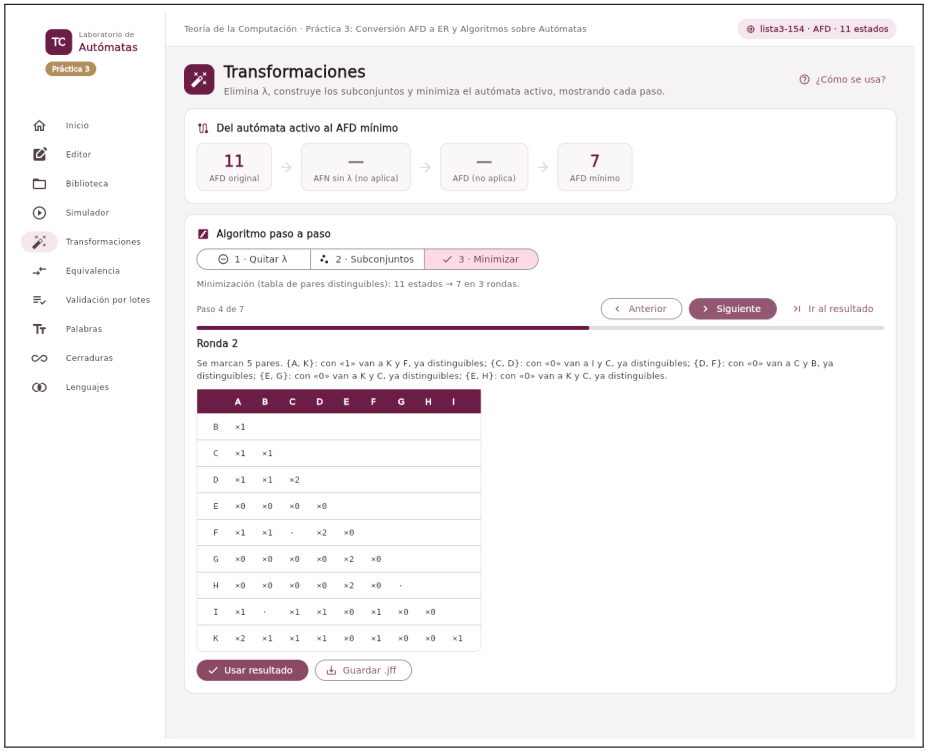


Figura 7. Minimización del AFD 154: tabla de pares en la ronda 2.

3.5. Validación de varias cadenas

En **Validación por lotes** se escribe o se carga un archivo de texto con una palabra por línea (las líneas vacías y las que empiezan con # se ignoran; λ es la palabra vacía). *Validar todo* muestra

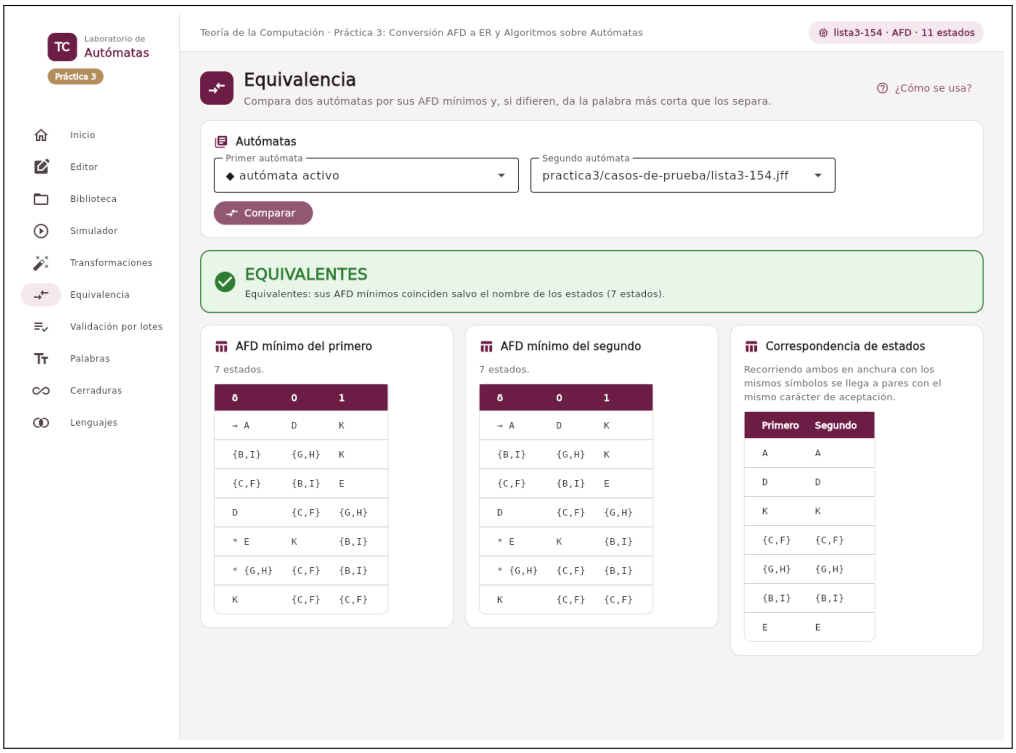


Figura 8. Equivalencia entre un autómata y su AFD mínimo, con la correspondencia de estados.

una tabla con el resultado de cada palabra y contadores de aceptadas, rechazadas y con error (por ejemplo, un símbolo fuera de Σ); *Guardar informe* escribe `salidas/informe-<autómata>.txt`. El archivo de ejemplo es `automatas/practica3/cadenas-ejemplo.txt`.

3.6. Pruebas automatizadas

`tests/test_algoritmos.py` (69 pruebas) comprueba, para cada uno de los 20 casos de prueba de `automatas/practica3/casos-de-prueba/`, que **el autómata original, el AFN sin λ, el AFD de subconjuntos y el AFD mínimo aceptan exactamente las mismas palabras**, y que el mínimo no tiene más estados que el AFD del que procede.

Longitud de la comparación. La comparación recorre *todas* las palabras de longitud $\leq$ L con $L = n_1 + n_2 - 2$, donde n_1 y n_2 son los estados del AFD de subconjuntos y del AFD mínimo una vez completados. La elección se justifica con el teorema de Moore: si dos AFD completos de n_1 y n_2 estados aceptan distinto alguna palabra, la más corta de ellas mide a lo sumo $n_1 + n_2 - 2$, porque el autómata producto distingue sus estados en a lo sumo ese número de rondas de refinamiento

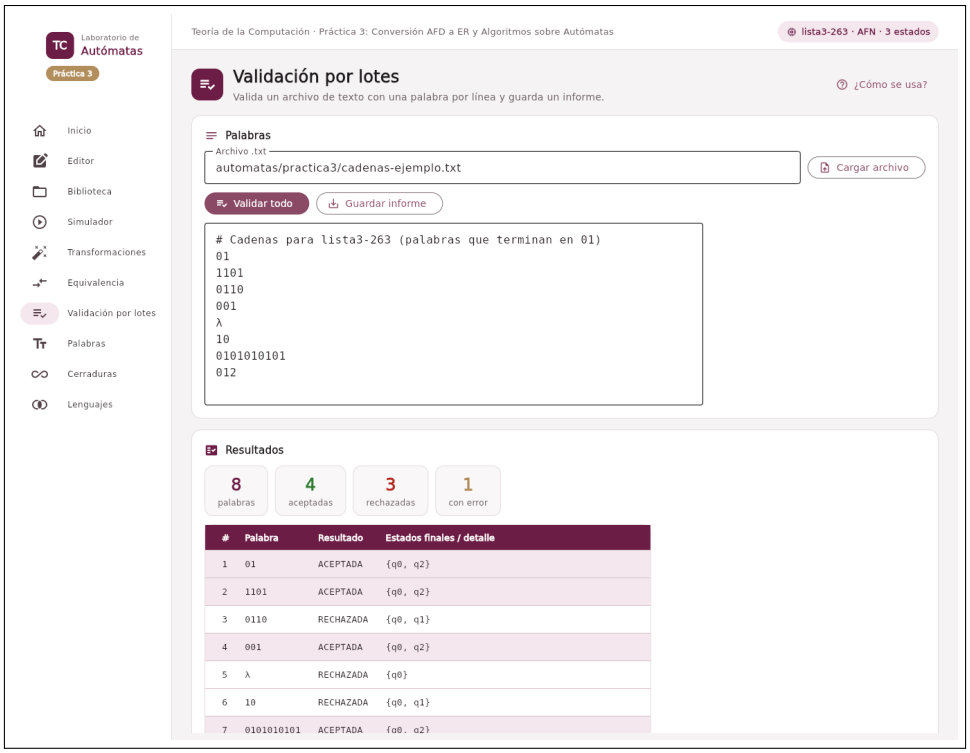


Figura 9. Validación por lotes de `cadenas-ejemplo.txt` con el AFN 263.

(Moore, 1956; Hopcroft et al., 2007, sec. 4.4). Con ese L la prueba exhaustiva no es un muestreo sino una demostración. Se usa además $L \geq 8$ para que los casos pequeños no queden triviales; el mayor L es 17 (ejercicio 195). Para que el costo no crezca con L, las palabras se recorren como un árbol, actualizando los estados activos símbolo a símbolo.

Otras pruebas comprueban los resultados conocidos de la Lista 3 (por ejemplo, que 154 se reduce a 7 estados y que tiene inaccesibles), que minimizar el mínimo no lo reduce, el número de rondas, que Δ' coincide con su definición en cada estado de 196 a 200, que sólo se generan subconjuntos alcanzables, el contraejemplo más corto y el informe del lote. La suite completa (176 aprobadas, 1 omitida que corresponde a la Práctica 4) se aprueba con Python 3.11, 3.12 y 3.13 en los contenedores y en GitHub Actions.

4. Ejercicio 3. Documentación de los algoritmos

En las estimaciones, $n = |Q|$, $k = |\Sigma|$ y m es el número de transiciones ($m \leq n^2(k + 1)$ en un AFN-λ).

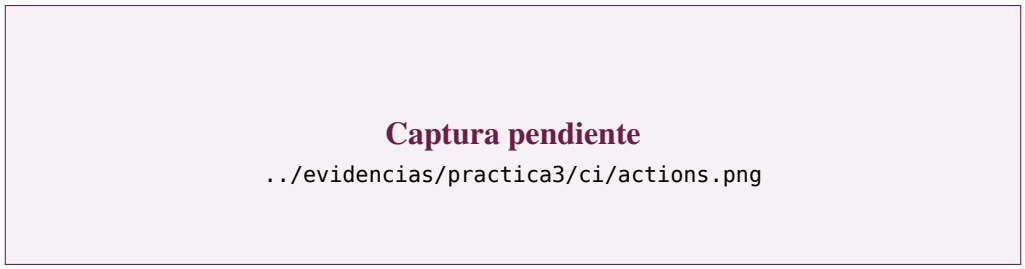


Figura 10. GitHub Actions con las tres versiones aprobadas (etiqueta `practica3`).

4.1. Cerradura λ y simulación directa

Problema. Dado un AFN-λ y un conjunto de estados S, hallar los estados alcanzables desde S sin consumir símbolos; con ello, decidir si el autómata acepta una palabra sin convertirlo.

Resultado teórico. $\hat{\delta}(q, \lambda) = \lambda\text{-cl}(q)$ y $\hat{\delta}(q, \omega a) = \lambda\text{-cl}(\bigcup_{p \in \hat{\delta}(q,\omega)} \delta(p, a))$; ω se acepta si $\hat{\delta}(q_0, \omega) \cap F \neq \varnothing$ (Hopcroft et al., 2007, sec. 2.5.3–2.5.4).

```
CIERRE(S)                                   # Automata.cierre
    visitados ← S; pila ← S
    mientras pila ≠ ∅: q ← pop(pila)
        para r ∈ δ(q, λ) − visitados: visitados ← visitados ∪ {r}; push(pila, r)
    devolver visitados

SIMULAR(ω)                                  # Simulacion.avanzar / completar
    S ← CIERRE({q0})
    para cada símbolo a de ω: S ← CIERRE(⋃_{q∈S} δ(q, a))     # Automata.transitar
    aceptar si S ∩ F ≠ ∅                                       #
↪   Automata.es_aceptacion
```

Ejemplo (196). A mano: λ-cl(A) = {A, B} porque A $\to\lambda$ B; λ-cl(B) = {B}, λ-cl(C) = {C}. Con ω = `01`: inicio {A, B}; con 0, δ(A, 0) $\cup$ δ(B, 0) = {A} $\cup$ {A, B} = {A, B}, cuyo cierre es {A, B}; con 1, δ(B, 1) = {C}; C $\in$ F, se acepta. El software da la misma tabla de cerraduras y la misma traza.

Pasos. Una cerradura revisa cada estado y cada arista λ una vez: O(n + m_λ). Cada paso de la simulación une hasta n conjuntos de hasta n estados y los cierra: O(n^2), y una palabra cuesta O($|\omega| \cdot n^2$).

4.2. Eliminación de transiciones λ

Problema. Dado un AFN-λ, obtener un AFN sin transiciones λ con el mismo lenguaje y los mismos estados.

Resultado teórico. Con Δ'(q, a) = λ-cl(δ(λ-cl(q), a)) y F' = {q : λ-cl(q) $\cap$ F $\neq \varnothing$}, el AFN (Q, Σ, Δ', q_0, F') reconoce el mismo lenguaje (Hopcroft et al., 2007, sec. 2.5.5; Sipser, 2013, sec. 1.2, demostración del teorema 1.39).

```
QUITAR-λ(N)                                  # QuitarLambda.resolver
    para q ∈ Q: C[q] ← CIERRE({q})           # etapa 1
    para q ∈ Q, a ∈ Σ: Δ'(q, a) ← CIERRE(⋃_{p∈C[q]} δ(p, a))   # etapa 2
    F' ← {q : C[q] ∩ F ≠ ∅}                  # etapa 3
    devolver (Q, Σ, Δ', q₀, F')              # etapa 4
```

Ejemplo (196). A mano: Δ'(A, 0) = λ-cl(δ({A, B}, 0)) = λ-cl({A, B}) = {A, B}, que añade B a δ(A, 0) = {A}; Δ'(A, 1) = λ-cl(δ({A, B}, 1)) = {C}, que no existía. B y C conservan sus transiciones, y como ninguna cerradura distinta de la de C contiene a C, F' = {C}. El software marca exactamente esos dos destinos añadidos («(+B)» y «(+C)») y reporta que ningún estado nuevo pasa a ser final.

Pasos. n cerraduras más, para cada uno de los n·k pares, una unión de hasta n conjuntos y un cierre: O(k·n³) en el peor caso.

4.3. Construcción de subconjuntos

Problema. Dado un AFN (o AFN-λ), obtener un AFD equivalente construyendo sólo los subconjuntos alcanzables.

Resultado teórico. Para todo AFN existe un AFD equivalente cuyos estados son subconjuntos de Q (Rabin & Scott, 1959, teorema 11; Hopcroft et al., 2007, sec. 2.3.5, teorema 2.11).

```
SUBCONJUNTOS(N)                              # Subconjuntos.resolver
    S₀ ← CIERRE({q₀}); descubiertos ← [S₀]; cola ← [S₀]
    mientras cola ≠ ∅:
        S ← desencolar(cola)                 # una etapa por fila
```

```
        para a ∈ Σ:
            T ← CIERRE(⋃_{q∈S} δ(q, a))     # Automata.transitar
            si T ∉ descubiertos: agregar T a descubiertos y a cola
            δ_D(S, a) ← T
    F_D ← {S ∈ descubiertos : S ∩ F ≠ ∅}
```

Ejemplo (192). A mano: A = {A}; con 0 va a {B} = B y con 1 a {B, C} = C. B: con 0 a {A, B} = D, con 1 a ∅. C: con 0 a δ(B, 0) ∪ δ(C, 0) = {A, B, C} = E. D: con 0 a {A, B} (D), con 1 a {B, C} (C). E: con 0 a E y con 1 a C. Son finales C y E porque contienen al estado final C. El software genera las cinco filas en el mismo orden y con los mismos nombres; JFLAP (*Convert to DFA*) da un AFD de cinco estados equivalente.

Pasos. A lo sumo 2^n subconjuntos, y por cada uno k transiciones de costo O(n²): O(k·n²·2^n) en el peor caso; en los casos de prueba se generaron entre 2 y 12 subconjuntos.

4.4. Minimización

Problema. Dado un AFD, obtener el AFD con el menor número de estados que reconoce el mismo lenguaje.

Resultado teórico. El AFD mínimo es único salvo el nombre de los estados y sus estados corresponden a las clases de la relación de Myhill-Nerode (Nerode, 1958); dos estados son equivalentes si ninguna palabra los distingue, y el algoritmo de llenado de tabla encuentra exactamente los pares distinguibles (Hopcroft et al., 2007, sec. 4.4.1–4.4.3).

```
MINIMIZAR(D)                                # Minimizacion.resolver
    eliminar estados inaccesibles           # Automata.alcanzables
    si δ es parcial: completar con un sumidero      # AFD.completar
    marcar {p, q} si exactamente uno está en F      # ronda 0
    repetir (ronda r = 1, 2, …):
        para cada par {p, q} sin marcar y a ∈ Σ:
            si {δ(p, a), δ(q, a)} está marcado: marcar {p, q} en la ronda r
    hasta que una ronda no marque nada
    clases ← pares sin marcar                       # Minimizacion._clases
    construir el AFD de clases (y retirar la del sumidero si se agregó)
```

Ejemplo (156). A mano: el único final es E, así que la ronda 0 marca los cuatro pares con E.

En la ronda 1, con el símbolo 1, A y C van a C mientras que B y D van a E; como {C, E} está marcado, se marcan {A, B}, {A, D}, {B, C} y {C, D}. La ronda 2 no marca nada. Quedan {A, C} y {B, D} sin marcar: el mínimo tiene tres estados, {A, C}, {B, D} y E. El software muestra la tabla triangular de las tres rondas con las mismas marcas y el mismo resultado.

Equivalencia. Dos AFD son equivalentes si sus mínimos coinciden salvo el nombre; `Equivalencia` lo decide recorriendo ambos mínimos en paralelo, y si no lo son, una búsqueda en anchura sobre el producto da la palabra más corta que los distingue (Hopcroft et al., 2007, sec. 4.4.2).

Pasos. Hay $n(n-1)/2$ pares y a lo sumo $n-2$ rondas útiles, cada una de costo $O(k \cdot n^2)$: $O(k \cdot n^3)$ en total. El algoritmo de Hopcroft baja el costo a $O(k \cdot n \log n)$ (Hopcroft, 1971). La prueba de isomorfismo cuesta $O(k \cdot n)$ y la del contraejemplo $O(k \cdot n_1 \cdot n_2)$.

4.5. Validación por lotes

Problema. Validar muchas palabras con un mismo autómata y conservar el resultado. No requiere teoría nueva: es SIMULAR(ω) para cada línea.

```
LOTE(N, texto)                              # Lote.validar_texto
    para cada línea no vacía que no empiece con #:
        ω ← (λ si la línea es «λ»; si no, la línea)
        registrar (número, ω, ACEPTADA/RECHAZADA/ERROR, estados finales)
    escribir informe                         # Lote.informe → Informe.guardar
```

Ejemplo. Con el AFN 263 (palabras que terminan en 01) y `cadenas-ejemplo.txt`: `01`, `1101`, `001` y `0101010101` se aceptan; `0110`, λ y `10` se rechazan, y `012` produce un error porque $2 \notin \Sigma$. Es lo que se obtiene a mano al revisar el sufijo de cada palabra.

Pasos. La suma, sobre las palabras, de $O(|\omega| \cdot n^2)$.

5. Conclusiones

Convertir los cinco AFD a expresiones mostró que la eliminación de estados es correcta en cualquier orden pero no igual de útil: eliminar primero el sumidero y los estados con menos conexiones produjo expresiones de entre 5 y poco más de 50 caracteres, mientras que el orden de Q llevó la de

140b a 234. Comparar los tres métodos sobre 142a y 138 dejó ver que la eliminación de estados y el lema de Arden son el mismo cálculo escrito como grafo o como sistema de ecuaciones, y que las expresiones $R_{ij}^{(k)}$, uniformes y por eso útiles para demostrar el teorema de Kleene, son las más largas de aplicar a mano.

En el software, dar a todos los algoritmos la misma forma —una clase con `resolver()` que devuelve el resultado y sus etapas— permitió que la interfaz los mostrara paso a paso con un único visor y que cada uno se probara sin abrirla. La tabla de pares distinguibles resultó la forma más clara de explicar la minimización ronda por ronda, y nombrar los subconjuntos con letras sin ocultar su contenido mantuvo legibles los AFD resultantes.

Las pruebas no se conformaron con muestrear: al comparar las cuatro versiones de cada caso sobre todas las palabras hasta la longitud $n_1 + n_2 - 2$ que da el teorema de Moore, la comprobación exhaustiva es también una demostración de equivalencia, y recorrer las palabras como un árbol la mantuvo por debajo de tres segundos en las tres versiones de Python.

Referencias

Arden, D. N. (1961). Delayed-logic and finite-state machines. *2nd Annual Symposium on Switching Circuit Theory and Logical Design*, 133-151. https://doi.org/10.1109/FOCS.1961.13

Hopcroft, J. E. (1971). An n log n algorithm for minimizing states in a finite automaton. En Z. Kohavi & A. Paz (Eds.), *Theory of Machines and Computations* (pp. 189-196). Academic Press. https://doi.org/10.1016/B978-0-12-417750-5.50022-1

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introduction to automata theory, languages, and computation* (3.ª ed.). Pearson Addison-Wesley.

Moore, E. F. (1956). Gedanken-experiments on sequential machines. En C. E. Shannon & J. McCarthy (Eds.), *Automata Studies* (pp. 129-153). Princeton University Press.

Nerode, A. (1958). Linear automaton transformations. *Proceedings of the American Mathematical Society*, *9*(4), 541-544. https://doi.org/10.1090/S0002-9939-1958-0135681-9

Rabin, M. O., & Scott, D. (1959). Finite automata and their decision problems. *IBM Journal of Research and Development*, *3*(2), 114-125. https://doi.org/10.1147/rd.32.0114

Rodger, S. H., & Finley, T. W. (2006). *JFLAP: An interactive formal languages and automata package*. Jones; Bartlett.

Schreiner, W. (2013). *Computability and complexity: Lecture notes* (Apuntes de curso). Research Institute for Symbolic Computation, Johannes Kepler University Linz. https://www3.risc.jku.at/people/schreine/courses/computability/

Sipser, M. (2013). *Introduction to the theory of computation* (3.ª ed.). Cengage Learning.